

MOD-05: React Hooks, Forms & Routing

Enterprise React Specialist Handbook · Principal Frontend Architect track, HCLSoftware

Sep 18, 2026 · @Someone

Chapter scope and standing conventions

MOD-04 established what a component is. MOD-05 is where a component acquires **memory, timing and a location in the application**. It is also the module where the runtime primitives from MOD-01 through MOD-03 stop being background knowledge and become the difference between an engineer who fixes hook bugs and one who reshuffles dependency arrays until the warning goes away.

ID	Subtopic	Downstream module that breaks without it
MOD-05.1	<code>useState</code> — snapshots, updaters, batching, lazy init	MOD-06.3 reducer mental model
MOD-05.2	<code>useEffect</code> — dependencies, cleanup, races, when not to	MOD-06.6 cancellation, MOD-07.2 Suspense
MOD-05.3	<code>useRef</code> — instance box, DOM refs, timers, <code>forwardRef</code>	MOD-07.3 virtualization, measurement
MOD-05.4	<code>useMemo</code> — referential stability, real cost	MOD-06.1 Context value identity
MOD-05.5	<code>useCallback</code> — handler stability, <code>React.memo</code> pairing	MOD-07.3 profiling-driven memoization
MOD-05.6	Custom hooks — extraction, return shape, composition	MOD-06.4 RTK Query hooks, org hook library
MOD-05.7	Forms — controlled vs uncontrolled, async submit, wizards	MOD-06.2 error boundaries around submission
MOD-05.8	React Router — nesting, params, guards, lazy boundaries	MOD-06.2 route error pages, MOD-07.1 code splitting

Standing conventions carried forward

1. **Strict TypeScript.** `strict: true`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes: true`, `any`, no silencing casts. Generic custom hooks are genuinely generic (`useResilientRequest<T>`), not `unknown` with a cast at the call site.
2. **Jest + React Testing Library.** Components tested through accessible roles and names; custom hooks tested with `renderHook` and `act`. Timers faked, `fetch` mocked, aborts asserted.
3. **WCAG 2.1 AA.** Labelling, focus management across step and route transitions, live-region announcements, and keyboard parity are verified in Part 6 rather than audited afterwards.
4. **APP-01** — *Enterprise Build Tooling and CI/CD Pipelines* remains appended after MOD-07.

Source alignment: built against the learner's handwritten notes dated 11/08/26 and 16/09/26, which already record the three parts of an effect — "(a) Setup function (b) Cleanup function: it undoes the setup (c) Dependency array: tells React when to re-run the setup" — and the `useState` → `render` → `useEffect` sequence with synchronization to an external system.

Prerequisite gate. MOD-02.1 (closures), MOD-01.2 (`Object.is` and reference identity) and MOD-02.6 (`AbortController`) must answer cold. This chapter does not re-teach them; it applies them.

Part 1 — High-level overview and architectural connections

Hooks are not features. They are a **positional storage protocol**: React keeps an ordered list of slots on each Fiber, and every hook call consumes the next slot in order. That single implementation fact is the reason for both Rules of Hooks — call them at the top level, never inside a branch or loop — because a conditional hook call shifts every subsequent slot and hands one hook another's state. The learner's source material states the rules; this module supplies the mechanism that makes them non-negotiable.

The eight subtopics

MOD-05.1 — `useState`. Snapshot semantics (established in MOD-04.4) plus the API depth: functional updaters, automatic batching, lazy initialization for expensive initial values, and immutable update patterns for nested objects and arrays.

MOD-05.2 — `useEffect` . The synchronization primitive, not a lifecycle hook. Dependency comparison, cleanup timing, StrictMode double-invocation, fetch race conditions, and — most important — the set of cases where an effect is the wrong tool entirely.

MOD-05.3 — `useRef` . A mutable box that survives renders and does **not** trigger one. Three distinct jobs: DOM element references, instance values like timer IDs and abort controllers, and previous-value storage. Plus `forwardRef` for exposing a child's DOM node through a design-system boundary.

MOD-05.4 — `useMemo` . Two legitimate purposes: caching a genuinely expensive computation, and preserving referential stability for a value that feeds a dependency array or a memoized child. Everything else is overhead with a maintenance cost.

MOD-05.5 — `useCallback` . `useMemo` specialized to functions. Its only real customers are `React.memo` children, effect dependency arrays, and the return values of custom hooks.

MOD-05.6 — Custom hooks. The extraction mechanism that replaced higher-order components and render props. The architectural content is the **return contract**: tuple versus object, and why an unstable object literal returned from a hook silently breaks every consumer's memoization.

MOD-05.7 — Forms. Controlled versus uncontrolled, the native Constraint Validation API, async submission state as a discriminated union, multi-step wizards, and the accessibility obligations that forms carry more heavily than any other UI.

MOD-05.8 — React Router. Route configuration, nested routes and `<Outlet />`, path and search params, programmatic navigation, role-checked protected routes, and lazy boundaries that MOD-07.1 will fill with code-split chunks.

The runtime grounding: three inheritances that decide everything

Closures (MOD-02.1) govern stale state in effects and callbacks. Every render creates a fresh set of functions, and each one closes over *that render's* props and state. An effect with `[]` captures the first render's values permanently. A `setInterval` callback created in that effect reads the first render's `count` forever, no matter how many times the component re-renders. This is not a React quirk — it is exactly the capture behaviour from MOD-02.1, made visible by a framework that creates a lot of closures. **The three fixes map directly onto MOD-02.1's lessons**: use a functional updater so the callback never needs to read the current value; declare the real dependency so a new closure is created when the value changes; or store the value in a ref so the closure reads a live box instead of a captured constant.

Object.is (MOD-01.2) controls the dependency array. React compares each dependency to its previous value with `Object.is`. Primitives compare by value and behave intuitively. Objects, arrays and functions compare by reference — so an object literal or arrow function declared in the component body is a **new value every render**, and any effect depending on it runs every render. This is the single most common cause of infinite effect loops, and it is why `useMemo` and `useCallback` exist at all: they are reference-stability tools, and their entire purpose is invisible to anyone who does not hold MOD-01.2.

AbortController (MOD-02.6) integrates into effect cleanup. The cleanup function is where MOD-02.6's cancellation discipline lands. An effect that fetches creates a controller, passes its signal to `fetch`, and returns `() => controller.abort()`. React guarantees the previous effect's cleanup runs **before** the next effect's setup, which is precisely the ordering that makes tenant-switch races impossible rather than merely unlikely. The `AbortError` must then be filtered out before any state write, exactly as in MOD-02.6 — a deliberate cancellation is not an incident.

Dependency topology

flowchart TD

```

A["MOD-05.1<br/>useState"] --> B["MOD-05.2<br/>useEffect"]
A --> C["MOD-05.3<br/>useRef"]
B --> C
B --> D["MOD-05.4<br/>useMemo"]
D --> E["MOD-05.5<br/>useCallback"]
C --> F["MOD-05.6<br/>Custom hooks"]
E --> F
F --> G["MOD-05.7<br/>Forms"]
F --> H["MOD-05.8<br/>React Router"]
G --> H
I["MOD-02.1<br/>Closures"] -.->|"stale capture"| B
J["MOD-01.2<br/>Object.is"] -.->|"dependency<br/>comparison"| B
J -.-> D
K["MOD-02.6<br/>AbortController"] -.->|"cleanup"| B
H --> L["MOD-06.2<br/>Route error pages"]
H --> M["MOD-07.1<br/>Lazy chunks"]

```

Note that `useEffect` sits at the centre with three dotted inputs. Every hard bug in this module passes through it, and each of the three has its root in an earlier chapter.

What this chapter defers

`useReducer` is deliberately held back to MOD-06.3, where Redux makes the reducer pattern worth the abstraction; introducing it here produces developers who reach for it in place of two `useState` calls. `useContext` is MOD-06.1. `useLayoutEffect` is named in the effect lifecycle diagram and applied in MOD-07.3 where measurement actually matters. `useTransition` and `useDeferredValue` belong with the concurrent-rendering work in MOD-07.

Part 2 — Production use cases and enterprise failure modes

MOD-05.1 — `useState`

Use case. A bulk-action toolbar on a job console that selects, deselects and inverts selection across a visible page of rows.

Failure mode: batched updates collapse. Three `setSelected(selected.concat(id))` calls in one handler all read the same snapshot, so only the last survives and two selections vanish. The operator selects five rows and sees three. The functional updater is not a style preference here; it is the only correct form when the next value derives from the previous.

Failure mode: expensive initializer on every render.

`useState(buildPermissionMatrix(roles))` calls the builder on **every** render and throws the result away after the first. On a 2,000-permission matrix that is tens of milliseconds per keystroke elsewhere in the page. `useState(() => buildPermissionMatrix(roles))` runs it once.

Failure mode: nested mutation. `config.quotas.seats = 50; setConfig(config)` mutates in place, `Object.is` reports no change, and nothing renders — then the mutation appears later attached to an unrelated update, making the bug look non-deterministic.

MOD-05.2 — `useEffect`

Use case. A live metrics tile polling tenant throughput every five seconds.

Failure mode: the stale-closure poll. An effect with `[]` sets an interval whose callback closes over the first render's `tenantId` and `interval`. The user switches tenant; the poll keeps requesting the old tenant's metrics with valid credentials, and the tile displays stale numbers the operator believes are live. Nothing errors, nothing logs. In a workload-scheduling product this means an operator watching a queue that is not the queue on their screen.

Failure mode: the fetch race on rapid navigation. The user clicks through three tenants in two seconds. Three requests are in flight; the slowest resolves last and paints tenant A's data under tenant C's header. Without `AbortController` plus an `AbortError` filter, this is a cross-tenant data-isolation failure, reportable under most enterprise contracts.

Failure mode: the infinite loop. `useEffect(() => { ... }, [filters])` where `filters` is an object literal built in the component body. `Object.is` sees a new reference every render, the effect runs, it sets state, the component re-renders, a new literal is built, the effect runs again. The tab pegs a CPU core and the API receives thousands of requests per minute from one browser.

Failure mode: the effect that should not exist. An effect that derives `filteredRows` from `rows` and writes it to state produces two renders per change, a frame of stale UI between them, and a second source of truth. The correct answer is a derivation during render. **The rule: if an effect's only job is to compute state from other state or props, delete it.**

MOD-05.3 — `useRef`

Use case. A debounced search box and an auto-refreshing audit log on the same screen.

Failure mode: the uncleared timer. A `setTimeout` id stored in a ref with no cleanup means that on unmount the timer still fires, calling a setter on a dead component and holding the whole closure — with its captured tenant data — alive. Navigate between twenty screens in a shift and the tab accumulates orphaned timers and megabytes of retained heap. The user reports that the console gets slow after lunch.

Failure mode: a ref used where state belonged. Storing the current filter in a ref means changing it updates the box but never re-renders, so the UI shows the previous filter's results. Developers then "fix" it with a forced re-render, which is a state variable wearing a disguise.

Failure mode: reading `ref.current` during render. A DOM ref is `null` on the first render and only populated after commit. Reading `inputRef.current.focus()` in the component body throws on mount — and works in development if a hot reload happens to skip the first render, which is how it reaches production.

MOD-05.4 — `useMemo`

Use case. A dashboard computing aggregate quota utilisation across 40 widgets.

Failure mode: memoizing everything. A team adopts `useMemo` for every derived value including `const label = \`${a} ${b}\``. Each wrapper allocates a dependency array and a closure and adds a comparison, which for a trivial computation costs more than

recomputing it. Across a large app this is measurable in initial load and in memory, and it makes every component harder to read.

Failure mode: memoizing with unstable dependencies. `useMemo(() => compute(config), [config])` where `config` is a fresh literal every render never hits the cache. The code looks optimized, is strictly slower than not memoizing, and passes review because reviewers see `useMemo` and assume performance.

Failure mode: `useMemo` as a cache. Using it to memoize an API response "so we do not refetch" misunderstands the primitive: React may discard memoized values at any time, and the memo is scoped to one component instance. It is a render-time optimization, never a data cache.

MOD-05.5 — `useCallback`

Use case. A virtualized grid of 5,000 rows, each wrapped in `React.memo`.

Failure mode: the memo defeated by one prop. Every row receives `onSelect={() => select(row.id)}` — a new function each render, so `React.memo` sees changed props and re-renders all 5,000 rows on every parent update. The team concludes memoization "does not work" and removes it, rather than stabilizing the handler.

Failure mode: `useCallback` everywhere. The opposite error. Wrapping handlers on plain DOM elements adds dependency arrays that go stale, produces bugs that look like unresponsive buttons, and buys nothing — React updates a stored reference with no DOM work. Premature `useCallback` has shipped more defects in enterprise React than it has prevented.

MOD-05.6 — Custom hooks

Use case. `useTenantFeatureFlags` published from a platform team to four product teams.

Failure mode: the unstable return object. The hook returns `{ flags, isLoading, refetch }` built as a fresh literal every render. Every consumer that puts the result in a dependency array or passes it to a memoized child re-renders continuously. One hook degrades four products, and each team debugs it independently because the cause is in someone else's package.

Failure mode: the hook that hides a fetch with no cancellation. A shared data hook without an abort path means every consumer inherits the tenant-switch race, and no consumer can fix it.

Failure mode: premature extraction. Extracting a "hook" used in exactly one place adds a file, an indirection and a test surface while removing the ability to read the component top to bottom. The extraction threshold is a second real consumer or a genuinely separable concern, not a line count.

MOD-05.7 — Forms

Use case. A tenant provisioning form with roughly 50 fields across identity, quotas, networking and billing.

Failure mode: the fully controlled 50-field form. Every keystroke calls `setState` at the form level, re-rendering all 50 fields plus their validation. On a mid-range corporate laptop typing feels laggy after about 25 fields, and users report "dropped characters" — the keystrokes are not dropped, the render is behind.

Failure mode: validation that only exists in JavaScript. Custom validation with no `aria-invalid`, no `aria-describedby` and no programmatic focus means a screen-reader user submits, hears nothing, and has no way to find which of 50 fields failed. This is an SC 3.3.1 failure and, for public-sector tenants, a procurement blocker.

Failure mode: double submission. No pending state on the submit button means a slow provisioning call gets clicked three times and creates three tenants. Submission state must be modelled explicitly, and a discriminated union makes the impossible states unrepresentable.

Failure mode: the wizard that loses focus. Advancing to step two leaves focus on the now-unmounted "Next" button, so focus falls to `<body>`. A keyboard user has to tab from the top of the document on every step, and a screen-reader user is never told the step changed.

MOD-05.8 — React Router

Use case. An admin console at `/tenants/:tenantId/settings` alongside a read-only auditor view.

Failure mode: the guard that renders before it redirects. A protected route that checks the role inside a `useEffect` and calls `navigate('/403')` has already **rendered the admin console for one frame** — and, worse, its child effects have already fired the privileged data fetch. The auditor sees a flash of the admin UI, and the network tab contains the response. Authorization must happen during render, before the protected subtree mounts, and must still be enforced server-side.

Failure mode: auth state read as "not authenticated" while loading. A guard that treats `user === null` as unauthenticated bounces every user to the login page on a hard refresh, because the session check has not resolved yet. The auth state must be a three-

state union — `loading`, `authenticated`, `anonymous` — not a nullable object.

Failure mode: search params duplicated into state. Copying `?page=2` into a `useState` means the back button changes the URL but not the UI. The URL is state; duplicating it guarantees divergence.

Failure mode: route transition with no focus or announcement. Navigating replaces the main content but leaves focus where it was. A screen-reader user hears nothing and has no indication the page changed, which fails SC 2.4.3 and makes the application effectively unusable without sight.

Part 3 — Architectural flowcharts and comparison tables

Diagram 1: the effect lifecycle and cleanup sequence

```

flowchart TD
    A["Render N<br/>component function runs,<br/>new closures created"] --> B["Reconcile + Commit<br/>DOM mutated"]
    B --> C["useLayoutEffect N<br/>runs BEFORE paint"]
    C --> D["Browser paints"]
    D --> E{"First render?"}
    E -- "Yes" --> F["Setup N runs"]
    E -- "No" --> G{"Object.is on every dep:<br/>all equal?"}
    G -- "Yes" --> H["SKIP: no cleanup,<br/>no setup"]
    G -- "No" --> I["Cleanup N-1 runs FIRST<br/>(abort, clearInterval,<br/>removeEventListener)"]
    I --> F
    F --> J["Effect N is live,<br/>holding render N's closure"]
    J --> K{"Next event?"}
    K -- "re-render" --> A
    K -- "unmount" --> L["Cleanup N runs"]
    L --> M["Component gone.<br/>No setter may fire<br/>after this point."]
  
```

Four guarantees carry the whole diagram, and each has a defect attached to ignoring it.

1. **Effects run after paint.** The user sees the committed output before the effect runs. Do not put layout measurement here — that is `useLayoutEffect`, which runs before paint and blocks it.
2. **Cleanup N-1 runs before setup N.** This ordering is what makes `AbortController` reliable for tenant switching: the previous request is aborted before the new one is issued, so it can never land last.
3. **Every dependency is compared with `Object.is`.** All equal means neither cleanup nor setup runs. One unstable object literal means both run on every single render.

4. **Cleanup runs on unmount.** Anything that can outlive the component — a timer, a listener, a subscription, an in-flight request — must be undone here, or it leaks and eventually writes to a dead component.

StrictMode in development deliberately runs setup, cleanup, then setup again on mount. An effect that survives that sequence correctly is an effect with honest cleanup. If a feature breaks under StrictMode, the effect is missing a teardown, not React misbehaving.

Diagram 2: route resolution and the protected-route guard

flowchart TD

```

A["Navigation: URL change"] --> B["Router matches<br/>the route tree"]
B --> C{"Match found?"}
C -- "No" --> D["404 route"]
C -- "Yes" --> E["Render layout route<br/>+ Outlet chain"]
E --> F["ProtectedRoute<br/>evaluates DURING RENDER"]
F --> G{"Auth status?"}
G -- "loading" --> H["Render skeleton.<br/>Do NOT redirect yet."]
G -- "anonymous" --> I["Navigate to /login<br/>replace + return-to state"]
G -- "authenticated" --> J{"Role satisfies<br/>requiredRole?"}
J -- "No" --> K["Navigate to /403<br/>replace"]
J -- "Yes" --> L["Render Outlet:<br/>protected subtree mounts"]
L --> M["Suspense boundary<br/>+ lazy chunk loads"]
M --> N["Child effects fire:<br/>privileged data fetched"]
N --> O["Focus moved to h1,<br/>route announced politely"]
H -. -> |"auth resolves"| G

```

The critical ordering. The guard returns a `<Navigate />` **during render**, before `<Outlet />` is reached. Nothing in the protected subtree mounts, so no privileged effect fires and no privileged request is issued. A guard implemented as a `useEffect` that calls `navigate()` runs *after* the subtree has already mounted and fetched — the auditor sees a flash of the admin console and the response is in the network tab.

The three-state auth rule. `loading` must be distinct from `anonymous`. Collapsing them bounces every authenticated user to the login page on a hard refresh, because the session check has not resolved yet. And in all cases this guard is a **UX affordance, not a security control** — the server enforces authorization independently, because anything in the browser can be bypassed.

Table 1: hook selection matrix

	useState	useRef	useMemo	useCallback	Custom hook
Stored where	Fiber state slot	Fiber ref slot (<code>.current</code>)	Fiber memo slot	Fiber memo slot	Composes the others
Triggers re-render on change	Yes	No	No (recompute only)	No	Depends on what it composes
Survives re-render	Yes	Yes	Value may be discarded	May be discarded	Yes
Readable during render	Yes	DOM refs: no (null until commit)	Yes	Yes	Yes
Main use case	Values the UI displays	DOM nodes, timer ids, abort controllers, previous values	Expensive computation; referential stability	Stable function identity	Reusing stateful logic across components
Primary misapplication	Storing derived values that should be computed	Storing something the UI must react to	Wrapping trivial computations; using it as a data cache	Wrapping every handler by reflex	Extracting after one use
Review question	"Can this be derived instead?"	"Does the UI need to react to this?"	"Did you measure it?"	"Is the consumer memoized?"	"Is there a second consumer?"

The decision path. Does the UI need to change when this value changes? Yes → `useState` . No, but it must survive renders → `useRef` . Is it derivable from existing state or props? → derive it during render, no hook at all. Is that derivation measurably expensive, or does it feed a dependency array or a memoized child? → `useMemo` , or `useCallback` if it is a function.

Table 2: form architecture matrix

	Fully controlled	Uncontrolled (refs + Constraint Validation)	Headless library (React Hook Form, TanStack Form)
Re-renders per keystroke	Whole form subtree	None	Only the field, via subscriptions
Performance at 50 fields	Degrades noticeably; typing lag	Flat	Flat

	Fully controlled	Uncontrolled (refs + Constraint Validation)	Headless library (React Hook Form, TanStack Form)
Validation timing	Any moment — on change, blur, submit	Browser-native on submit; manual otherwise	Configurable per field and per event
Cross-field and async validation	Straightforward	Awkward — state lives in the DOM	First-class, including debounced async
Conditional fields and wizards	Straightforward	Painful	First-class
Built-in accessibility	Everything hand-wired	Browser supplies some messaging, but it is not customizable	Error and <code>aria</code> wiring supplied, still needs verification
Testability	Simple, fully in React	Requires DOM interaction	Good, with library test helpers
Type safety	Manual	Weak — values arrive as strings	Strong; schema-inferred types via Zod or Valibot
Enterprise fit	Small forms (under ~10 fields), or any field needing live cross-field feedback	Simple search and filter bars; a login form	Default for anything provisioning-scale: 20+ fields, wizards, async validation

The position this handbook takes. Build LAB-05 controlled and hand-wired, because the learner must understand what a form library does before delegating it. Then, in production, adopt a headless library for anything over roughly 15 fields and keep controlled components for the small, interactive cases. A fully controlled 50-field form is a known performance failure, and a hand-rolled validation layer is a known accessibility failure — both are solved problems, and solving them again is not architecture.

Part 4 — Low-level deep dive and interactive coding lab

Strict TypeScript throughout. Work each debug challenge before reading its diagnosis.

MOD-05.1 — useState

Mechanism. `useState` consumes the next state slot on the Fiber and returns `[value, setter]`. The value is a constant for the render. The setter enqueues an update and schedules work; React 18 batches every update in the same synchronous block into one re-render. The lazy form — `useState(() => expensive())` — calls the initializer only on the first render; `useState(expensive())` calls it on every render and discards the result.

```
// Bad – snapshot collisions, eager initializer, nested mutation
const [selected, setSelected] = useState<string[]>([]);
const [matrix] = useState(buildPermissionMatrix(roles)); // runs EVERY render

const selectThree = (ids: readonly string[]): void => {
  ids.forEach(id => setSelected(selected.concat(id))); // all read one
  snapshot
};

const setSeats = (n: number): void => {
  config.quotas.seats = n; // mutation: same reference
  setConfig(config);      // Object.is -> no change -> no render
};

// Production – Set for membership, functional updaters, lazy init, immutable
// nesting
const [selected, setSelected] = useState<ReadonlySet<string>>(() => new
Set());
const [matrix] = useState<PermissionMatrix>(() =>
buildPermissionMatrix(roles));

const selectMany = (ids: readonly string[]): void => {
  setSelected(current => {
    const next = new Set(current); // copy, then mutate the copy
    ids.forEach(id => next.add(id));
    return next;                  // new reference: Object.is sees a
    change
  });
};

const setSeats = (n: number): void => {
  setConfig(current => ({
    ...current,
    quotas: { ...current.quotas, seats: n }, // every level on the path
    copied
  }));
};
```

Note the `Set` copy: a `Set` mutated in place is the same reference and will not render. Copy, mutate the copy, return it.

Type Guard Note. `useState<Tenant | null>(null)` forces every consumer to narrow before use, which is correct. `useState<Tenant>()` infers `Tenant | undefined` and quietly makes the undefined case invisible at call sites — prefer the explicit union.

Debug challenge DBG-05.1

```
const [filters, setFilters] = useState<Filters>({ status: 'all', page: 1 });

const nextPage = (): void => {
  setFilters({ ...filters, page: filters.page + 1 });
  setFilters({ ...filters, status: 'active' });
};
```

► Reveal diagnosis

The page never advances. Both spreads read the same `filters` snapshot, so the second update overwrites the first and the committed value is `{ status: 'active', page: 1 }`. Use a single functional updater: `setFilters(c => ({ ...c, page: c.page + 1, status: 'active' }))`, or two chained functional updaters. The defect only appears when two updates touch the same state object in one handler, which is why it survives review.

MOD-05.2 — `useEffect`

Mechanism. An effect synchronizes a component with an **external system**: a subscription, a timer, a network request, a browser API, a non-React widget. It is not a lifecycle callback. React compares each dependency with `Object.is`; if any differ, it runs the previous cleanup and then the new setup. On unmount it runs the final cleanup.

```
// Bad — stale closure, no cleanup, no cancellation, no error discrimination
useEffect(() => {
  setInterval(() => {
    fetch(`/api/tenants/${tenantId}/metrics`)
      .then(r => r.json())
      .then(setMetrics);
  }, 5000);
}, []); // captures the first render's tenantId forever

// Production — real dependencies, abort on teardown, AbortError filtered
useEffect(() => {
  const controller = new AbortController();
  let cancelled = false;
```

```

const poll = async (): Promise<void> => {
  try {
    const res = await fetch(`/api/tenants/${tenantId}/metrics`, {
      signal: controller.signal,
    });
    if (!res.ok) throw new HttpError(res.status);
    const data = (await res.json()) as TenantMetrics;
    if (!cancelled) setMetrics(data);
  } catch (error) {
    if (error instanceof Error && error.name === 'AbortError') return; //
deliberate
    if (!cancelled) setError(toMessage(error));
  }
};

void poll();
const id = window.setInterval(() => void poll(), intervalMs);

return () => {
  cancelled = true;
  controller.abort(); // MOD-02.6, applied at teardown
  window.clearInterval(id);
};
}, [tenantId, intervalMs]);

```

Both guards are needed. `controller.abort()` stops the network work; `cancelled` closes the window between a resolved promise and the abort landing.

When NOT to use an effect — the four cases that account for most misuse:

Instinct	Correct answer
Derive state from props or other state	Compute it during render
Reset state when a prop changes	<code>key={id}</code> on the component (MOD-04.6)
Respond to a user action	Do it in the event handler
Cache a computed value	<code>useMemo</code> , not effect-plus-state

Debug challenge DBG-05.2

```
function JobList({ filters }: { readonly filters: JobFilters }):
React.ReactElement {
  const [jobs, setJobs] = useState<readonly Job[]>([]);

  useEffect(() => {
    void loadJobs(filters).then(setJobs);
  }, [filters]);

  return <Table rows={jobs} />;
}

// Call site
<JobList filters={{ status: 'running', tenantId }} />
```

► Reveal diagnosis

An infinite request loop. The call site builds a fresh object literal on every parent render, so `Object.is(prev, next)` is always `false`, the effect runs every time, each response sets state, and the cycle repeats. The fix is at the **call site**, not in the effect: memoize the object (`useMemo(() => ({ status, tenantId }), [status, tenantId])`), or pass primitives and depend on those. Silencing the exhaustive-deps warning converts a loop into a stale closure, which is worse because it is silent.

MOD-05.3 — useRef

Mechanism. `useRef(initial)` returns a stable object `{ current }` for the component's whole lifetime. Writing `.current` never triggers a render and is not tracked by React. Three distinct jobs: a DOM handle (populated after commit, `null` during the first render), an instance value (timer ids, controllers, flags), and previous-value storage updated inside an effect.

```
// Bad – leaked timer, ref read during render, ref where state belonged
function SearchBox(): React.ReactElement {
  const timerRef = useRef<number | undefined>(undefined);
  const inputRef = useRef<HTMLInputElement>(null);
  const queryRef = useRef(''); // UI must react to this: wrong
  tool

  inputRef.current!.focus(); // null on first render: throws

  const onChange = (e: React.ChangeEvent<HTMLInputElement>): void => {
    queryRef.current = e.target.value; // no re-render: results go stale
    timerRef.current = window.setTimeout(() => search(queryRef.current),
300);
  }; // never cleared: leaks on unmount
```

```

    return <input ref={inputRef} onChange={onChange} />;
  }

  // Production – state for UI, ref for the timer, focus in an effect, cleanup
  function SearchBox({ onSearch }: { readonly onSearch: (q: string) => void })
  {
    const [query, setQuery] = useState('');
    const timerRef = useRef<number | undefined>(undefined);
    const inputRef = useRef<HTMLInputElement>(null);

    useEffect(() => {
      inputRef.current?.focus(); // after commit: the node exists
    }, []);

    useEffect(() => {
      timerRef.current = window.setTimeout(() => onSearch(query), 300);
      return () => window.clearTimeout(timerRef.current);
    }, [query, onSearch]);

    return (
      <>
        <label htmlFor="job-search">Search jobs</label>
        <input
          id="job-search"
          ref={inputRef}
          value={query}
          onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
            setQuery(e.target.value)}
        />
      </>
    );
  }

```

forwardRef exposes a child's DOM node through a design-system boundary, which is how a form library focuses your `<TextField>` on validation failure:

```

interface TextFieldProps {
  readonly label: string;
  readonly error?: string;
}

export const TextField = forwardRef<HTMLInputElement, TextFieldProps>(
  function TextField({ label, error }, ref) {
    const id = useId();
    return (
      <div>

```

```

    <label htmlFor={id}>{label}</label>
    <input
      id={id}
      ref={ref}
      aria-invalid={error !== undefined}
      aria-describedby={error !== undefined ? `${id}-err` : undefined}
    />
    {error !== undefined && <span id={`${id}-err`} role="alert">{error}</span>}
  </div>
);
},
);

```

Debug challenge DBG-05.3

```

function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T>();
  ref.current = value;
  return ref.current;
}

```

► Reveal diagnosis

It always returns the **current** value, never the previous one. The assignment happens during render, before the return, so the caller reads what it just wrote. Defer the write to an effect, which runs after the render has consumed the old value:

```

function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T | undefined>(undefined);
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current; // read during render = the last committed render's value
}

```

Writing a ref during render is also a purity violation (MOD-04.2) and can misbehave under Concurrent rendering, where a render may be discarded after the write.

MOD-05.4 — useMemo

Mechanism. `useMemo(factory, deps)` stores the factory's result and recomputes only when a dependency changes by `Object.is`. It has a cost: a closure, a dependency array, and a comparison every render. It pays for itself in exactly two situations — a genuinely expensive computation, or a value whose **reference identity** must stay stable for a dependency array or a memoized child.

```
// Bad – memoizing nothing, and memoizing against an unstable dependency
const label = useMemo(() => `${tenant.name} (${tenant.region})`, [tenant]);
const report = useMemo(() => buildReport(config), [config]); // config is a
fresh literal

// Production – derive cheap values plainly; memoize real cost and real
identity
const label = `${tenant.name} (${tenant.region})`; // cheaper than
memoizing

const report = useMemo(
  () => buildReport(rows, thresholds), // O(n log n) over ~50k rows:
  measured
  [rows, thresholds], // both stable references from props
);

// Stability for a Context value – MOD-06.1's most common defect, prevented
here
const tenantContext = useMemo<TenantContextValue>(
  () => ({ tenantId, region, permissions }),
  [tenantId, region, permissions],
);
```

Debug challenge DBG-05.4

```
function QuotaPanel({ tenantId }: { readonly tenantId: string }):
React.ReactElement {
  const options = useMemo(() => ({ tenantId, includeArchived: false }), []);
  const { data } = useQuotaSummary(options);
  return <Summary data={data} />;
}
```

► Reveal diagnosis

The memo is stable but **wrong**: with an empty dependency array it freezes `tenantId` at the first render, so switching tenants keeps requesting the original tenant's quotas. This is the stale-closure bug wearing a `useMemo`. The array must list `tenantId`. Note the asymmetry worth internalizing: an over-broad dependency array causes extra work (a performance problem, visible), while an under-broad one causes stale values (a correctness problem, silent). When in doubt, over-include.

MOD-05.5 — useCallback

Mechanism. `useCallback(fn, deps)` is `useMemo(() => fn, deps)`. It returns the same function reference until a dependency changes. It does nothing for performance on its own — its value is entirely in what *consumes* the reference.

```
// Bad — useCallback on a plain element (pointless), inline handler on a memo
child (harmful)
const onChange = useCallback((e: React.ChangeEvent<HTMLInputElement>) => {
  setQuery(e.target.value);
}, []); // buys nothing: <input> is not memoized

{rows.map(row => (
  <MemoRow key={row.id} row={row} onSelect={() => select(row.id)} /> //
  defeats memo
))}

// Production — stable handler taking the id as an argument, memo intact
const handleSelect = useCallback((id: string): void => {
  setSelected(current => {
    const next = new Set(current);
    if (next.has(id)) next.delete(id);
    else next.add(id);
    return next;
  });
}, []); // functional updater means no dependency on `selected`

{rows.map(row => (
  <MemoRow key={row.id} row={row} onSelect={handleSelect} /> // stable
  reference
))}
```

The functional updater is what makes the empty dependency array honest. Writing `[selected]` would produce a new handler on every selection and defeat the memo it was created to serve.

Debug challenge DBG-05.5

```
function Toolbar({ onRefresh }: { readonly onRefresh: () => void }):
  React.ReactElement {
  const handleRefresh = useCallback(() => {
    onRefresh();
  }, []);

  useEffect(() => {
    const id = window.setInterval(handleRefresh, 30_000);
    return () => window.clearInterval(id);
  }, [handleRefresh]);

  return <button type="button" onClick={handleRefresh}>Refresh</button>;
}
```

► Reveal diagnosis

`handleRefresh` closes over the first render's `onRefresh` and never updates, because the dependency array omits it. When the parent passes a new `onRefresh` — a different tenant's refresh, say — both the button and the 30-second interval keep calling the original. The interval is especially damaging: it looks like it is working, at the correct cadence, against the wrong target. Add `onRefresh` to the array. Then note what follows: if the parent creates `onRefresh` inline, the callback is recreated every render, the effect tears down and re-creates the interval every render, and the timer never reaches 30 seconds. Stability is a **chain** — one unstable link upstream makes every downstream memoization useless.

MOD-05.6 — Custom hooks

Mechanism. A custom hook is a function whose name begins with `use` and which calls other hooks. It shares *logic*, never state: two components calling the same hook get two independent sets of slots. The architectural decisions are the extraction threshold and the return contract.

Return shape: a **tuple** when the consumer will rename the values and there are at most two or three (`const [value, setValue] = useToggle()`); an **object** when there are several or the set may grow, because adding a field is then non-breaking. Whichever you choose, the reference must be stable.

```
// Bad – unstable return object breaks every consumer's memoization
export function useTenantFlags(tenantId: string) {
  const [flags, setFlags] = useState<Flags>({});
  const [isLoading, setLoading] = useState(true);
  const refetch = () => void load(tenantId).then(setFlags);
  return { flags, isLoading, refetch }; // new object AND new function each
```

```

render
}

// Production – stable callbacks, stable return object, exported result type
export interface UseTenantFlagsResult {
  readonly flags: Readonly<Flags>;
  readonly isLoading: boolean;
  readonly refetch: () => void;
}

export function useTenantFlags(tenantId: string): UseTenantFlagsResult {
  const [flags, setFlags] = useState<Flags>({});
  const [isLoading, setLoading] = useState(true);

  const refetch = useCallback(() : void => {
    setLoading(true);
    void load(tenantId)
      .then(setFlags)
      .finally(() => setLoading(false));
  }, [tenantId]);

  useEffect(() => { refetch(); }, [refetch]);

  return useMemo(
    () => ({ flags, isLoading, refetch }),
    [flags, isLoading, refetch],
  );
}

```

Enterprise hook guidelines. Name it for what it gives, not how it works (`useTenantFlags` , not `useFlagsFetcher`). Export a result interface so consumers can type against it. Every hook that starts something must stop it. A hook in a shared package is an API: breaking its return shape breaks four teams at once, so treat additive change as the only safe change.

Debug challenge DBG-05.6

```

export function useInterval(callback: () => void, delayMs: number): void {
  useEffect(() => {
    const id = window.setInterval(callback, delayMs);
    return () => window.clearInterval(id);
  }, [callback, delayMs]);
}

// Consumer
useInterval(() => refreshMetrics(tenantId), 5000);

```

► Reveal diagnosis

The interval is destroyed and re-created on **every render** of the consumer, because the inline arrow is a new reference each time. With a 5-second delay and a component that re-renders more often than that, the callback may never fire at all. The standard fix stores the callback in a ref so the effect does not depend on its identity:

```
export function useInterval(callback: () => void, delayMs: number | null):
void {
  const savedRef = useRef(callback);

  useEffect(() => { savedRef.current = callback; }, [callback]);

  useEffect(() => {
    if (delayMs === null) return; // null pauses the
interval
    const id = window.setInterval(() => savedRef.current(), delayMs);
    return () => window.clearInterval(id);
  }, [delayMs]);
}
```

The ref gives the interval a **live** reference rather than a captured one — the third of the three closure fixes from Part 1, and the reason `useRef` is taught before custom hooks.

MOD-05.7 — Forms

Mechanism. A controlled input's `value` comes from React state and every keystroke round-trips through a re-render. An uncontrolled input keeps its value in the DOM, read via a ref or `FormData` on submit. The native Constraint Validation API (`required`, `pattern`, `min`, `checkValidity()`, `validationMessage`) provides real validation for free and works before JavaScript loads.

```
// Bad — one state object for 50 fields, no submission state, clickable
during submit
const [values, setValues] = useState<Record<string, string>>({});

<input value={values.name ?? ''} onChange={e => setValues({ ...values, name:
e.target.value })} />
<button onClick={submit}>Create tenant</button>

// Production — submission state as a discriminated union; native validation
first
type SubmitState =
  | { readonly kind: 'idle' }
  | { readonly kind: 'submitting' }
```

```

    | { readonly kind: 'error'; readonly message: string }
    | { readonly kind: 'success'; readonly tenantId: string };

const [submitState, setSubmitState] = useState<SubmitState>({ kind: 'idle'
});

const handleSubmit = async (event: React.FormEvent<HTMLFormElement>):
Promise<void> => {
    event.preventDefault();
    if (submitState.kind === 'submitting') return;          // structural double-
submit guard

    const form = event.currentTarget;
    if (!form.checkValidity()) {                            // native validation
first
        form.reportValidity();
        return;
    }

    setSubmitState({ kind: 'submitting' });
    try {
        const tenantId = await createTenant(new FormData(form));
        setSubmitState({ kind: 'success', tenantId });
    } catch (error) {
        setSubmitState({ kind: 'error', message: toMessage(error) });
    }
};

<button type="submit" disabled={submitState.kind === 'submitting'}>
    {submitState.kind === 'submitting' ? 'Creating...' : 'Create tenant'}
</button>
{submitState.kind === 'error' && <p role="alert">{submitState.message}</p>}

```

The union makes "submitting and error at once" unrepresentable, and the `disabled` binding follows from the same source of truth as the label.

Debug challenge DBG-05.7

```

const [email, setEmail] = useState<string | undefined>(undefined);

<input
    type="email"
    value={email}
    onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
setEmail(e.target.value)}
/>

```

► Reveal diagnosis

React logs *"A component is changing an uncontrolled input to be controlled."* `value={undefined}` makes the input uncontrolled on the first render; the first keystroke supplies a string and switches it to controlled. In practice this loses the first character in some browsers and breaks form reset. A controlled input's value must never be `undefined` or `null` — initialize to `''` and type the state `string`. The same applies to `checked`.

MOD-05.8 — React Router

Mechanism. The router matches the URL against a nested route tree. Parent routes render layout plus `<Outlet />`, where the matched child renders. `useParams` reads path segments, `useSearchParams` reads and writes the query string as URL state, and `useNavigate` navigates imperatively. Guards belong in the route tree as layout routes, so protection is structural rather than repeated per page.

```
// Bad — guard in an effect: the subtree already mounted and fetched
function AdminRoute(): React.ReactElement {
  const { user } = useAuth();
  useEffect(() => {
    if (user?.role !== 'Admin') navigate('/403');
  }, [user]);
  return <Outlet />;
}

// Production — evaluated during render, three-state auth, return-to-preserved
type AuthState =
  | { readonly status: 'loading' }
  | { readonly status: 'anonymous' }
  | { readonly status: 'authenticated'; readonly user: User };

interface ProtectedRouteProps {
  readonly requiredRole?: Role;
}

export function ProtectedRoute({ requiredRole }: ProtectedRouteProps):
React.ReactElement {
  const auth = useAuth();
  const location = useLocation();

  if (auth.status === 'loading') {
    return <RouteSkeleton aria-busy="true" />;           // NOT a redirect
  }
}
```

```

    if (auth.status === 'anonymous') {
      return <Navigate to="/login" replace state={{ from: location }} />;
    }
    if (requiredRole !== undefined && !hasRole(auth.user, requiredRole)) {
      return <Navigate to="/403" replace />;
    }
    return <Outlet />;
  }

  // Route tree: protection is structural, not repeated per page
  <Routes>
    <Route path="/" element={<AppLayout />}>
      <Route index element={<Dashboard />} />
      <Route element={<ProtectedRoute requiredRole="Admin" />}>
        <Route path="tenants/:tenantId/settings" element={<TenantSettings />} />
      />
      <Route path="tenants/new" element={<TenantProvisioningWizard />} />
    </Route>
    <Route path="403" element={<Forbidden />} />
    <Route path="*" element={<NotFound />} />
  </Route>
</Routes>

```

`replace` matters: without it the back button returns the user to the page that just bounced them, producing a redirect loop.

Search params are state. Read them from the URL rather than copying them into `useState`:

```

const [searchParams, setSearchParams] = useSearchParams();
const page = Number(searchParams.get('page') ?? '1');

const goToPage = (next: number): void => {
  setSearchParams(current => {
    const params = new URLSearchParams(current);
    params.set('page', String(next));
    return params;
  }, { replace: true }); // pagination should not flood browser history
};

```

Debug challenge DBG-05.8

```
function TenantSettings(): React.ReactElement {
  const { tenantId } = useParams();
  const [settings, setSettings] = useState<Settings | null>(null);

  useEffect(() => {
    void fetchSettings(tenantId).then(setSettings);
  }, [tenantId]);

  return <SettingsForm settings={settings} />;
}
```

► Reveal diagnosis

Three defects. First, `useParams()` returns `Record<string, string | undefined>`, so `tenantId` is `string | undefined`; under `strict` this either fails to compile or silently fetches `/api/tenants/undefined`. Validate it and render a not-found state when it is missing. Second, there is no cancellation: navigating between tenants quickly lets a slow response for tenant A overwrite tenant B's settings — the cross-tenant race from Part 2, requiring `AbortController` plus an `AbortError` filter. Third, `settings` is `null` on the first render and `SettingsForm` almost certainly does not handle that, so this either crashes or renders a form full of `undefined` values; model the load as a discriminated union (`loading | error | ready`) rather than a nullable object.

Coding Lab LAB-05 — `TenantProvisioningWizard.tsx`

Brief. A three-step accessible provisioning wizard in strict TypeScript, behind an Admin route guard, built on two reusable custom hooks. Step 1 collects tenant identity with live slug-availability checking; step 2 collects quota limits; step 3 confirms and submits.

Hook 1 — useDebounceValue<T>

```
// hooks/useDebounceValue.ts
import { useEffect, useState } from 'react';

/**
 * Returns `value` delayed by `delayMs`. The timer is cleared on every change
 * and on unmount, so no update can land after teardown (MOD-03.4, MOD-05.3).
 */
export function useDebounceValue<T>(value: T, delayMs: number): T {
  const [debounced, setDebounce] = useState<T>(value);

  useEffect(() => {
    const id = window.setTimeout(() => setDebounce(value), delayMs);
    return () => window.clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}
```

The entire debounce lives in the cleanup: each new `value` clears the pending timer before scheduling the next, which is the MOD-03.4 implementation expressed through the effect lifecycle.

Hook 2 — useResilientRequest<T>

```
// hooks/useResilientRequest.ts
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';

export class HttpError extends Error {
  constructor(readonly status: number, message?: string) {
    super(message ?? `Request failed with status ${status}`);
    this.name = 'HttpError';
  }
}

export type RequestState<T> =
  | { readonly status: 'idle' }
  | { readonly status: 'loading' }
  | { readonly status: 'success'; readonly data: T }
  | { readonly status: 'error'; readonly error: RequestError };

export type RequestError =
  | { readonly kind: 'http'; readonly status: number; readonly message: string }
  | { readonly kind: 'network'; readonly message: string }
```

```

    | { readonly kind: 'parse'; readonly message: string };

export interface UseResilientRequestOptions {
  readonly enabled?: boolean;
  readonly timeoutMs?: number;
}

export interface UseResilientRequestResult<T> {
  readonly state: RequestState<T>;
  readonly refetch: () => void;
}

export function useResilientRequest<T>(
  url: string | null,
  options: UseResilientRequestOptions = {},
): UseResilientRequestResult<T> {
  const { enabled = true, timeoutMs = 10_000 } = options;
  const [state, setState] = useState<RequestState<T>>({ status: 'idle' });
  const [nonce, setNonce] = useState(0);
  const controllerRef = useRef<AbortController | null>(null);

  const refetch = useCallback(() : void => setNonce(n => n + 1), []);

  useEffect(() => {
    if (url === null || !enabled) {
      setState({ status: 'idle' });
      return;
    }

    const controller = new AbortController();
    controllerRef.current = controller;
    const timeoutId = window.setTimeout(() => controller.abort(), timeoutMs);
    setState({ status: 'loading' });

    const run = async (): Promise<void> => {
      try {
        const res = await fetch(url, { signal: controller.signal });
        if (!res.ok) throw new HttpError(res.status);
        const data = (await res.json()) as T;
        setState({ status: 'success', data });
      } catch (error) {
        // A deliberate abort is not an incident (MOD-02.6)
        if (error instanceof Error && error.name === 'AbortError') return;
        setState({ status: 'error', error: toRequestError(error) });
      } finally {
        window.clearTimeout(timeoutId);
      }
    };
  }, [url, enabled, timeoutMs]);
}

```

```

    }
  };

  void run();

  // Cleanup N-1 runs BEFORE setup N: the old request can never land last
  return () => {
    window.clearTimeout(timeoutId);
    controller.abort();
  };
}, [url, enabled, timeoutMs, nonce]);

// Stable return object: consumers can safely memoize against it (MOD-05.6)
return useMemo(() => ({ state, refetch }), [state, refetch]);
}

function toRequestError(error: unknown): RequestError {
  if (error instanceof HttpError) {
    return { kind: 'http', status: error.status, message: error.message };
  }
  if (error instanceof SyntaxError) {
    return { kind: 'parse', message: 'The server returned a malformed
response.' };
  }
  return {
    kind: 'network',
    message: error instanceof Error ? error.message : 'Network request
failed.',
  };
}

```

Note there is no `any` and no cast except the unavoidable `as T` on the parsed body, which is the one boundary where the compiler genuinely cannot know the shape. In production that line is where a Zod or Valibot schema belongs, turning the assertion into a validated parse.

The wizard

```

// TenantProvisioningWizard.tsx
import { useCallback, useEffect, useId, useRef, useState } from 'react';
import { useDebounceValue } from './hooks/useDebounceValue';
import { useResilientRequest } from './hooks/useResilientRequest';

// ----- Domain -----

export interface TenantIdentity {

```

```

    readonly name: string;
    readonly slug: string;
    readonly region: Region;
  }

export interface TenantQuotas {
  readonly seats: number;
  readonly storageGb: number;
  readonly concurrentJobs: number;
}

export type Region = 'us-east-1' | 'eu-west-1' | 'ap-south-1';

interface SlugAvailability {
  readonly available: boolean;
}

type StepIndex = 0 | 1 | 2;

const STEP_TITLES = ['Tenant identity', 'Quota limits', 'Confirmation'] as
const;

type SubmitState =
  | { readonly kind: 'idle' }
  | { readonly kind: 'submitting' }
  | { readonly kind: 'error'; readonly message: string }
  | { readonly kind: 'success'; readonly tenantId: string };

export interface TenantProvisioningWizardProps {
  readonly onComplete: (tenantId: string) => void;
  readonly createTenant: (
    identity: TenantIdentity,
    quotas: TenantQuotas,
  ) => Promise<string>;
}

// ----- Pure validation: unit-testable, no React -----

export function validateIdentity(v: TenantIdentity): ReadonlyMap<string,
string> {
  const errors = new Map<string, string>();
  if (v.name.trim().length < 3) {
    errors.set('name', 'Tenant name must be at least 3 characters.');
```

```

hyphens.');
```

```

    }
    return errors;
  }

export function validateQuotas(v: TenantQuotas): ReadonlyMap<string, string>
{
  const errors = new Map<string, string>();
  // `>= 0` not truthiness: 0 is a legitimate value (MOD-01.5)
  if (!Number.isInteger(v.seats) || v.seats < 1) {
    errors.set('seats', 'Seats must be a whole number of at least 1.');
```

```

  }
  if (v.storageGb < 0) {
    errors.set('storageGb', 'Storage cannot be negative.');
```

```

  }
  if (v.concurrentJobs < 0) {
    errors.set('concurrentJobs', 'Concurrent jobs cannot be negative.');
```

```

  }
  return errors;
}

// ----- Component -----

export function TenantProvisioningWizard({
  onComplete,
  createTenant,
}: TenantProvisioningWizardProps): React.ReactElement {
  const headingId = useId();
  const [step, setStep] = useState<StepIndex>(0);
  const [identity, setIdentity] = useState<TenantIdentity>({
    name: '',
    slug: '',
    region: 'us-east-1',
  });
  const [quotas, setQuotas] = useState<TenantQuotas>({
    seats: 1,
    storageGb: 0,
    concurrentJobs: 0,
  });
  const [showErrors, setShowErrors] = useState(false);
  const [submitState, setSubmitState] = useState<SubmitState>({ kind: 'idle'
});

  // Focus recovery: the step heading receives focus after every transition
  const headingRef = useRef<HTMLHeadingElement>(null);
  const isFirstRender = useRef(true);

```

```

useEffect(() => {
  if (isFirstRender.current) {
    isFirstRender.current = false;    // do not steal focus on mount
    return;
  }
  headingRef.current?.focus();
}, [step]);

// Live slug availability: debounced, cancelled, and never fired on an
invalid slug
const debouncedSlug = useDebounceValue(identity.slug, 400);
const slugIsWellFormed = /^[a-z0-9-]{3,32}$/.test(debouncedSlug);
const { state: slugState } = useResilientRequest<SlugAvailability>(
  slugIsWellFormed ? `/api/tenants/slug-available?
slug=${encodeURIComponent(debouncedSlug)}` : null,
  { enabled: step === 0 },
);

// Derived during render – never stored (MOD-04.4)
const identityErrors = validateIdentity(identity);
const quotaErrors = validateQuotas(quotas);
const slugTaken = slugState.status === 'success' &&
!slugState.data.available;
const canAdvance =
  step === 0
    ? identityErrors.size === 0 && !slugTaken && slugState.status !==
'loading'
    : step === 1
      ? quotaErrors.size === 0
      : true;

const currentErrors = step === 0 ? identityErrors : quotaErrors;

const goNext = useCallback(() : void => {
  setShowErrors(true);
  if (!canAdvance) return;
  setShowErrors(false);
  setStep(current => (current === 2 ? 2 : ((current + 1) as StepIndex)));
}, [canAdvance]);

const goBack = useCallback(() : void => {
  setShowErrors(false);
  setStep(current => (current === 0 ? 0 : ((current - 1) as StepIndex)));
}, []);

```

```

const handleSubmit = async (event: React.FormEvent<HTMLFormElement>):
Promise<void> => {
  event.preventDefault();
  if (step !== 2 || submitState.kind === 'submitting') return; // double-
submit guard

  setSubmitState({ kind: 'submitting' });
  try {
    const tenantId = await createTenant(identity, quotas);
    setSubmitState({ kind: 'success', tenantId });
    onComplete(tenantId);
  } catch (error) {
    setSubmitState({
      kind: 'error',
      message: error instanceof Error ? error.message : 'Provisioning
failed.',
    });
  }
};

const errorFor = (field: string): string | undefined =>
  showErrors ? currentErrors.get(field) : undefined;

return (
  <form onSubmit={handleSubmit} aria-labelledby={headingId} noValidate>
    /* Progress is announced, not conveyed by position alone */
    <p role="status" aria-live="polite">
      Step {step + 1} of 3: {STEP_TITLES[step]}
    </p>

    <h2 id={headingId} ref={headingRef} tabIndex={-1}>
      {STEP_TITLES[step]}
    </h2>

    {step === 0 && (
      <fieldset>
        <legend>Tenant identity</legend>

        <Field
          label="Tenant name"
          value={identity.name}
          error={errorFor('name')}
          onChange={name => setIdentity(c => ({ ...c, name })))}
        />

        <Field

```

```

        label="Slug"
        value={identity.slug}
        error={errorFor('slug') ?? (slugTaken ? 'That slug is already
taken.' : undefined)}
        hint="Lowercase letters, numbers and hyphens."
        onChange={slug => setIdentity(c => ({ ...c, slug })))}
      />

      <p role="status" aria-live="polite">
        {slugState.status === 'loading' && 'Checking slug availability...'}
        {slugState.status === 'success' &&
          (slugState.data.available ? 'Slug is available.' : 'Slug is
taken.'))}
        {slugState.status === 'error' &&
          'Could not check slug availability. You can continue and we
will verify on submit.'}
      </p>

      <RegionSelect
        value={identity.region}
        onChange={region => setIdentity(c => ({ ...c, region })))}
      />
    </fieldset>
  )}

  {step === 1 && (
    <fieldset>
      <legend>Quota limits</legend>
      <NumberField
        label="Seats"
        value={quotas.seats}
        error={errorFor('seats')}
        onChange={seats => setQuotas(c => ({ ...c, seats })))}
      />
      <NumberField
        label="Storage (GB)"
        value={quotas.storageGb}
        error={errorFor('storageGb')}
        onChange={storageGb => setQuotas(c => ({ ...c, storageGb })))}
      />
      <NumberField
        label="Concurrent jobs"
        value={quotas.concurrentJobs}
        error={errorFor('concurrentJobs')}
        onChange={concurrentJobs => setQuotas(c => ({ ...c,
concurrentJobs })))}

```

```

        />
      </fieldset>
    )}

    {step === 2 && (
      <dl>
        <dt>Name</dt><dd>{identity.name}</dd>
        <dt>Slug</dt><dd>{identity.slug}</dd>
        <dt>Region</dt><dd>{identity.region}</dd>
        <dt>Seats</dt><dd>{quotas.seats}</dd>
        <dt>Storage</dt><dd>{quotas.storageGb} GB</dd>
        <dt>Concurrent jobs</dt><dd>{quotas.concurrentJobs}</dd>
      </dl>
    )}

    {submitState.kind === 'error' && <p role="alert">{submitState.message}
  </p>}}

  <div>
    {step > 0 && (
      <button type="button" onClick={goBack}>Back</button>
    )}
    {step < 2 && (
      <button type="button" onClick={goNext}>Next</button>
    )}
    {step === 2 && (
      <button type="submit" disabled={submitState.kind === 'submitting'}>
        {submitState.kind === 'submitting' ? 'Creating...' : 'Create
tenant'}
      </button>
    )}
  </div>
</form>
);
}

// ----- Field primitives -----

interface FieldProps {
  readonly label: string;
  readonly value: string;
  readonly error?: string;
  readonly hint?: string;
  readonly onChange: (value: string) => void;
}

```

```

function Field({ label, value, error, hint, onChange }: FieldProps):
  React.ReactElement {
  const id = useId();
  const errorId = `${id}-error`;
  const hintId = `${id}-hint`;
  const describedBy = [hint !== undefined ? hintId : null, error !==
undefined ? errorId : null]
    .filter((x): x is string => x !== null)
    .join(' ');

  return (
    <div>
      <label htmlFor={id}>{label}</label>
      {hint !== undefined && <span id={hintId}>{hint}</span>}
      <input
        id={id}
        type="text"
        value={value}
        aria-invalid={error !== undefined}
        aria-describedby={describedBy === '' ? undefined : describedBy}
        onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
onChange(e.target.value)}
      />
      {error !== undefined && <span id={errorId} role="alert">{error}</span>}
    </div>
  );
}

interface NumberFieldProps {
  readonly label: string;
  readonly value: number;
  readonly error?: string;
  readonly onChange: (value: number) => void;
}

function NumberField({ label, value, error, onChange }: NumberFieldProps):
  React.ReactElement {
  const id = useId();
  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input
        id={id}
        type="number"
        value={value}
        aria-invalid={error !== undefined}

```

```

        aria-describedby={error !== undefined ? `${id}-error` : undefined}
        onChange={(e: React.ChangeEvent<HTMLInputElement>) => {
            const next = Number(e.target.value);
            if (!Number.isNaN(next)) onChange(next);
        }}
    />
    {error !== undefined && <span id={` ${id}-error`} role="alert">{error}</span>}
</div>
);
}

```

Accessibility decisions worth naming. The step heading carries `tabIndex={-1}` and receives focus on every transition, so a keyboard user lands at the top of the new step and a screen-reader user hears its name — the fix for the lost-focus failure in Part 2. The `isFirstRender` ref prevents focus being stolen on mount, which would be hostile if the wizard is embedded in a longer page. Progress is announced through a polite live region rather than conveyed by visual position. Every error is tied to its input with `aria-describedby` and marked `role="alert"`, and `aria-invalid` is driven by the same derived value as the message. `noValidate` disables the browser bubble so the custom messages are the single source of truth — a deliberate trade, acceptable only because every field is wired correctly.

Route integration

```

// routes.tsx
<Routes>
  <Route path="/" element={<AppLayout />}>
    <Route index element={<Dashboard />} />
    <Route element={<ProtectedRoute requiredRole="Admin" />}>
      <Route
        path="tenants/new"
        element={
          <TenantProvisioningWizard
            createTenant={createTenant}
            onComplete={id => navigate(`/tenants/${id}`)}
          />
        }
      />
    </Route>
    <Route path="403" element={<Forbidden />} />
  </Route>
</Routes>

```

The wizard never checks a role itself. Authorization is structural, enforced by the route tree before the wizard mounts, so the privileged slug-availability request is never issued for a non-admin.

RTL and Jest suite

```
import { render, screen, waitFor } from '@testing-library/react';
import { renderHook, act } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { MemoryRouter, Route, Routes } from 'react-router-dom';
import { TenantProvisioningWizard, validateQuotas } from
'./TenantProvisioningWizard';
import { useDebouncedValue } from './hooks/useDebouncedValue';
import { useResilientRequest } from './hooks/useResilientRequest';
import { ProtectedRoute } from './ProtectedRoute';

const available = { ok: true, status: 200, json: async () => ({ available:
true }) };

beforeEach(() => {
  global.fetch = jest.fn().mockResolvedValue(available) as unknown as typeof
fetch;
});
afterEach(() => jest.restoreAllMocks());

// ----- Pure validation -----

describe('validateQuotas', () => {
  it('accepts 0 for storage and jobs but not for seats', () => {
    const errors = validateQuotas({ seats: 1, storageGb: 0, concurrentJobs: 0
});
    expect(errors.size).toBe(0);
    expect(validateQuotas({ seats: 0, storageGb: 0, concurrentJobs: 0
}).has('seats'))
      .toBe(true);
  });
});

// ----- Hook: useDebouncedValue -----

describe('useDebouncedValue', () => {
  beforeEach(() => jest.useFakeTimers());
  afterEach(() => jest.useRealTimers());

  it('emits only the final value after the delay', () => {
    const { result, rerender } = renderHook(
```

```

    ({ value }) => useDebounceValue(value, 400),
    { initialProps: { value: 'a' } },
  );
  rerender({ value: 'ac' });
  rerender({ value: 'acm' });
  rerender({ value: 'acme' });
  expect(result.current).toBe('a'); // nothing emitted yet
  act(() => { jest.advanceTimersByTime(399); });
  expect(result.current).toBe('a');
  act(() => { jest.advanceTimersByTime(1); });
  expect(result.current).toBe('acme'); // one emission, the last
value
});

it('leaves no pending timer after unmount', () => {
  const { unmount, rerender } = renderHook(
    ({ value }) => useDebounceValue(value, 400),
    { initialProps: { value: 'a' } },
  );
  rerender({ value: 'ab' });
  unmount();
  expect(jest.getTimerCount()).toBe(0);
});
});

// ----- Hook: useResilientRequest -----

describe('useResilientRequest', () => {
  it('ABORTS the in-flight request on unmount', async () => {
    let capturedSignal: AbortSignal | undefined;
    global.fetch = jest.fn((_url, init?: RequestInit) => {
      capturedSignal = init?.signal ?? undefined;
      return new Promise(() => {}); // never resolves
    }) as unknown as typeof fetch;

    const { unmount } = renderHook(() => useResilientRequest<unknown>
('/api/x'));
    expect(capturedSignal?.aborted).toBe(false);
    unmount();
    expect(capturedSignal?.aborted).toBe(true);
  });

  it('aborts the previous request when the url changes (no race)', async ()
=> {
    const signals: AbortSignal[] = [];
    global.fetch = jest.fn((_url, init?: RequestInit) => {

```

```

    if (init?.signal) signals.push(init.signal);
    return new Promise(() => {});
  }) as unknown as typeof fetch;

  const { rerender } = renderHook(
    ({ url }) => useResilientRequest<unknown>(url),
    { initialProps: { url: '/api/tenants/a' } },
  );
  rerender({ url: '/api/tenants/b' });
  // Cleanup N-1 runs before setup N: the first signal is already aborted
  expect(signals[0]?.aborted).toBe(true);
  expect(signals[1]?.aborted).toBe(false);
});

it('never surfaces an AbortError as an error state', async () => {
  const { result, unmount } = renderHook(() => useResilientRequest<unknown>(
    '/api/x'));
  unmount();
  expect(result.current.state.status).not.toBe('error');
});

it('discriminates an HTTP failure from a network failure', async () => {
  global.fetch = jest.fn().mockResolvedValue({ ok: false, status: 503 }) as
  unknown as typeof fetch;
  const { result } = renderHook(() => useResilientRequest<unknown>(
    '/api/x'));
  await waitFor(() => expect(result.current.state.status).toBe('error'));
  const { state } = result.current;
  expect(state.status === 'error' && state.error.kind).toBe('http');
});

it('returns a STABLE object across renders with unchanged state', () => {
  const { result, rerender } = renderHook(() =>
  useResilientRequest<unknown>(null));
  const first = result.current;
  rerender();
  expect(result.current).toBe(first); // consumers can memoize against
  it
  });
});

// ----- Wizard -----

const renderWizard = (createTenant = jest.fn().mockResolvedValue('t-1')) => {
  const onComplete = jest.fn();
  const utils = render(

```

```

    <TenantProvisioningWizard createTenant={createTenant} onComplete=
{onComplete} />,
  );
  return { ...utils, createTenant, onComplete };
};

const fillIdentity = async (user: ReturnType<typeof userEvent.setup>) => {
  await user.type(screen.getByLabelText('Tenant name'), 'Acme EU');
  await user.type(screen.getByLabelText('Slug'), 'acme-eu');
};

describe('TenantProvisioningWizard', () => {
  it('renders step 1 with accessible names and an announced position', () =>
  {
    renderWizard();
    expect(screen.getByRole('heading', { name: 'Tenant identity'
})).toBeInTheDocument();
    expect(screen.getByRole('status')).toHaveTextContent('Step 1 of 3');
    expect(screen.getByLabelText('Tenant name')).toBeInTheDocument();
  });

  it('blocks advancing while step 1 is invalid and reports the reason', async
  () => {
    const user = userEvent.setup();
    renderWizard();
    await user.click(screen.getByRole('button', { name: 'Next' }));
    expect(screen.getByRole('heading', { name: 'Tenant identity'
})).toBeInTheDocument();
    expect(screen.getByLabelText('Tenant name')).toHaveAttribute('aria-
invalid', 'true');
    expect(screen.getByLabelText('Tenant name')).toHaveAccessibleDescription(
      /at least 3 characters/i,
    );
  });

  it('advances through all three steps', async () => {
    const user = userEvent.setup();
    renderWizard();
    await fillIdentity(user);
    await waitFor(() =>
      expect(screen.getByRole('button', { name: 'Next' })).toBeEnabled(),
    );
    await user.click(screen.getByRole('button', { name: 'Next' }));
    expect(screen.getByRole('heading', { name: 'Quota limits'
})).toBeInTheDocument();
  });
});

```

```

    await user.click(screen.getByRole('button', { name: 'Next' }));
    expect(screen.getByRole('heading', { name: 'Confirmation'
  })).toBeInTheDocument();
    expect(screen.getByRole('status')).toHaveTextContent('Step 3 of 3');
  });

  it('MOVES FOCUS to the step heading on every transition', async () => {
    const user = userEvent.setup();
    renderWizard();
    await fillIdentity(user);
    await waitFor(() =>
      expect(screen.getByRole('button', { name: 'Next' })).toBeEnabled(),
    );
    await user.click(screen.getByRole('button', { name: 'Next' }));
    expect(screen.getByRole('heading', { name: 'Quota limits'
  })).toHaveFocus();

    await user.click(screen.getByRole('button', { name: 'Back' }));
    expect(screen.getByRole('heading', { name: 'Tenant identity'
  })).toHaveFocus();
  });

  it('does NOT steal focus on mount', () => {
    renderWizard();
    expect(screen.getByRole('heading', { name: 'Tenant identity'
  })).not.toHaveFocus();
  });

  it('DEBOUNCES slug checking: many keystrokes, ONE request', async () => {
    jest.useFakeTimers();
    const user = userEvent.setup({ advanceTimers: jest.advanceTimersByTime
  });
    renderWizard();
    await user.type(screen.getByLabelText('Slug'), 'acme-eu');
    expect(global.fetch).not.toHaveBeenCalled();
    act(() => { jest.advanceTimersByTime(400); });
    await waitFor(() => expect(global.fetch).toHaveBeenCalledTimes(1));
    jest.useRealTimers();
  });

  it('does not check availability for a malformed slug', async () => {
    jest.useFakeTimers();
    const user = userEvent.setup({ advanceTimers: jest.advanceTimersByTime
  });
    renderWizard();
    await user.type(screen.getByLabelText('Slug'), 'AB'); // too short,

```

```

uppercase
  act(() => { jest.advanceTimersByTime(400); });
  expect(global.fetch).not.toHaveBeenCalled();
  jest.useRealTimers();
});

it('blocks advancing when the slug is taken and says so', async () => {
  global.fetch = jest.fn().mockResolvedValue({
    ok: true, status: 200, json: async () => ({ available: false }),
  }) as unknown as typeof fetch;
  const user = userEvent.setup();
  renderWizard();
  await fillIdentity(user);
  await waitFor(() =>

expect(screen.getByLabelText('Slug')).toHaveAccessibleDescription(/already
taken/i),
  );
  await user.click(screen.getByRole('button', { name: 'Next' }));
  expect(screen.getByRole('heading', { name: 'Tenant identity'
})).toBeInTheDocument();
});

it('aborts the in-flight slug check on unmount', async () => {
  let signal: AbortSignal | undefined;
  global.fetch = jest.fn((_u, init?: RequestInit) => {
    signal = init?.signal ?? undefined;
    return new Promise(() => {});
  }) as unknown as typeof fetch;

  const user = userEvent.setup();
  const { unmount } = renderWizard();
  await user.type(screen.getByLabelText('Slug'), 'acme-eu');
  await waitFor(() => expect(signal).toBeDefined());
  unmount();
  expect(signal?.aborted).toBe(true);
});

it('guards against double submission', async () => {
  const user = userEvent.setup();
  let resolve!: (id: string) => void;
  const createTenant = jest.fn(() => new Promise<string>(r => { resolve =
r; }));
  renderWizard(createTenant);

  await fillIdentity(user);

```

```

    await waitFor(() => expect(screen.getByRole('button', { name: 'Next'
    })).toBeEnabled()));
    await user.click(screen.getByRole('button', { name: 'Next' }));
    await user.click(screen.getByRole('button', { name: 'Next' }));

    const submit = screen.getByRole('button', { name: 'Create tenant' });
    await user.click(submit);
    expect(screen.getByRole('button', { name: 'Creating...' })).toBeDisabled();
    await user.click(screen.getByRole('button', { name: 'Creating...' }));
    expect(createTenant).toHaveBeenCalledTimes(1);

    act(() => resolve('t-1'));
  });

  it('announces a submission failure through an alert', async () => {
    const user = userEvent.setup();
    const createTenant = jest.fn().mockRejectedValue(new Error('Region at
capacity'));
    renderWizard(createTenant);

    await fillIdentity(user);
    await waitFor(() => expect(screen.getByRole('button', { name: 'Next'
    })).toBeEnabled()));
    await user.click(screen.getByRole('button', { name: 'Next' }));
    await user.click(screen.getByRole('button', { name: 'Next' }));
    await user.click(screen.getByRole('button', { name: 'Create tenant' }));

    expect(await screen.findByRole('alert')).toHaveTextContent('Region at
capacity');
  });

  it('is fully operable by keyboard', async () => {
    const user = userEvent.setup();
    renderWizard();
    await user.tab();
    expect(screen.getByLabelText('Tenant name')).toHaveFocus();
    await user.keyboard('Acme EU');
    await user.tab();
    expect(screen.getByLabelText('Slug')).toHaveFocus();
  });
});

// ----- Route guard -----

const renderAt = (auth: AuthState, path = '/tenants/new') =>
  render(

```

```

<AuthContext.Provider value={auth}>
  <MemoryRouter initialEntries={[path]}>
    <Routes>
      <Route element={}<ProtectedRoute requiredRole="Admin" />}>
        <Route path="tenants/new" element={}<div>ADMIN CONSOLE</div>} />
      </Route>
      <Route path="403" element={}<div>FORBIDDEN</div>} />
      <Route path="login" element={}<div>LOGIN</div>} />
    </Routes>
  </MemoryRouter>
</AuthContext.Provider>,
);

describe('ProtectedRoute', () => {
  it('redirects a non-admin to /403 WITHOUT rendering the console', () => {
    renderAt({ status: 'authenticated', user: { id: 'u1', role: 'Auditor' }
  });
    expect(screen.getByText('FORBIDDEN')).toBeInTheDocument();
    expect(screen.queryByText('ADMIN CONSOLE')).not.toBeInTheDocument();
  });

  it('does NOT fire the protected data request for a non-admin', () => {
    renderAt({ status: 'authenticated', user: { id: 'u1', role: 'Auditor' }
  });
    expect(global.fetch).not.toHaveBeenCalled(); // guard ran during render
  });

  it('renders the console for an admin', () => {
    renderAt({ status: 'authenticated', user: { id: 'u2', role: 'Admin' } });
    expect(screen.getByText('ADMIN CONSOLE')).toBeInTheDocument();
  });

  it('sends an anonymous user to /login rather than /403', () => {
    renderAt({ status: 'anonymous' });
    expect(screen.getByText('LOGIN')).toBeInTheDocument();
  });

  it('shows a skeleton while auth is loading instead of bouncing to login',
  () => {
    renderAt({ status: 'loading' });
    expect(screen.queryByText('LOGIN')).not.toBeInTheDocument();
    expect(screen.queryByText('ADMIN CONSOLE')).not.toBeInTheDocument();
    expect(screen.getByRole('status')).toHaveAttribute('aria-busy', 'true');
  });
});

```

The four tests that matter most are the abort-on-unmount test, the abort-on-url-change test, the focus-on-step-transition test, and the guard test asserting that `fetch` was never called for a non-admin. Each maps to a Part 2 failure that is invisible in manual QA: a leak, a cross-tenant race, an accessibility break, and a privilege leak.

Part 5 — Active recall flashcards

Review on a 1 / 3 / 7 / 21-day schedule. **Tier A** covers hook invariants and effect cleanup and gates MOD-06. **Tier B** is optimization and routing. **Tier C** is forms and edge cases.

#	Tier	Prompt	Answer
1	A	Why must hooks be called unconditionally at the top level?	React stores them in an ordered list of slots per Fiber. A conditional call shifts every later slot and hands one hook another's state.
2	A	In what order do cleanup and setup run when a dependency changes?	Cleanup N-1 runs first , then setup N. That ordering is what makes <code>AbortController</code> reliable for tenant switching.
3	A	What does React compare dependencies with?	<code>Object.is</code> . Primitives by value; objects, arrays and functions by reference — so a fresh literal re-runs the effect every render.
4	A	Why does an effect with <code>[]</code> read stale state?	Its closure captured the first render's values and is never recreated. Fix with a functional updater, real dependencies, or a ref.
5	A	Name the four cases where an effect is the wrong tool.	Deriving state from props or state; resetting state on a prop change (use <code>key</code>); responding to a user action (use the handler); caching a computed value (use <code>useMemo</code>).
6	A	What must every effect that starts something do?	Return a cleanup that stops it — abort, <code>clearInterval</code> , <code>clearTimeout</code> , <code>unsubscribe</code> , <code>removeEventListener</code> .

#	Tier	Prompt	Answer
7	A	Why does StrictMode run setup, cleanup, setup on mount?	To prove the effect's teardown is honest. A feature that breaks under it is missing cleanup, not suffering a React bug.
8	A	Does writing <code>ref.current</code> trigger a re-render?	No. Refs are untracked. If the UI must react to a value, it belongs in state.
9	B	When does <code>useMemo</code> actually pay for itself?	A measurably expensive computation, or a value whose reference identity feeds a dependency array or a memoized child. Nothing else.
10	B	What is <code>useCallback</code> in terms of <code>useMemo</code> ?	<code>useMemo(() => fn, deps)</code> . Useless alone; valuable only to a <code>React.memo</code> child, an effect dependency, or a hook's return contract.
11	B	Over-broad or under-broad dependency array — which is worse?	Under-broad. It causes silent stale values. Over-broad causes extra work, which is visible. When in doubt, over-include.
12	B	Why must a guard redirect during render, not in an effect?	An effect runs after the subtree has already mounted and fired its privileged requests. The user sees a flash of the console and the data is in the network tab.
13	B	Why must auth state be three-state?	<code>loading</code> collapsed into <code>anonymous</code> bounces every authenticated user to login on a hard refresh, before the session check resolves.
14	B	Where does pagination state belong?	The URL, via <code>useSearchParams</code> . Copying it into <code>useState</code> breaks the back button and guarantees divergence.
15	B	Why does an inline callback break <code>useInterval</code> ?	A new reference each render re-runs the effect, destroying and re-creating the interval so it may never fire. Store the callback in a ref.

#	Tier	Prompt	Answer
16	C	Why does a custom hook returning <code>{ a, b }</code> break consumers?	A fresh object every render. Any consumer memoizing against it re-renders continuously. Wrap the return in <code>useMemo</code> with stable members.
17	C	Tuple or object return for a custom hook?	Tuple for two or three values the consumer will rename; object when there are several or the set may grow, since adding a field stays non-breaking.
18	C	What is the extraction threshold for a custom hook?	A second real consumer, or a genuinely separable concern. Not a line count, and never a single use.
19	C	Why does <code>value={undefined}</code> break a controlled input?	It starts uncontrolled and switches to controlled on the first keystroke, losing a character and breaking reset. Initialize to <code>''</code> .
20	C	Why does a fully controlled 50-field form lag?	Every keystroke re-renders the whole form subtree plus validation. Use a headless form library above roughly 15 fields.
21	C	What must happen on a step or route transition?	Move focus to the new heading (<code>tabIndex=-1</code>) and announce the change politely. Otherwise keyboard and screen-reader users are stranded.
22	C	How do you make double submission structurally impossible?	Model submission as a discriminated union and return early when <code>kind === 'submitting'</code> , with <code>disabled</code> bound to the same value.

Part 6 — Enterprise production readiness checklist

No PR touching hooks, forms or routing merges until every applicable line is verified. Reviewers cite the line in the review comment.

Hook rule compliance and dependency arrays

- ☐ Every hook is called unconditionally at the top level; none inside a branch, loop or nested function

- ☐ `eslint-plugin-react-hooks` runs in CI with `exhaustive-deps` set to **error**, not warn
- ☐ No `eslint-disable` on a dependency array; the dependency is either genuinely stable or the code is restructured
- ☐ Object and array dependencies are stable references, memoized at their source rather than inside the consumer
- ☐ Functional updaters used wherever the next value derives from the previous, so the setter is not a dependency
- ☐ Expensive `useState` initial values use the lazy form `useState(() => ...)`
- ☐ Controlled inputs never receive `undefined` or `null` as `value` or `checked`

Effect cleanup and async safety

- ☐ Every effect that starts something returns a cleanup that stops it
- ☐ Every fetch inside an effect passes an `AbortController` signal and aborts on teardown
- ☐ `AbortError` is filtered before any state write or telemetry call
- ☐ Every timer and interval id is cleared on teardown and on dependency change
- ☐ Every subscription and listener is removed; `{ signal }` preferred for listeners
- ☐ No effect exists solely to derive state from props or other state
- ☐ No effect synchronizes state to props; the component resets with `key` instead
- ☐ Feature verified under StrictMode: mount, unmount, remount produces no duplicate requests, no doubled timers and no console warnings
- ☐ No setter can fire after unmount; verified by test, not by inspection
- ☐ Refs are never written or read for DOM nodes during render

Custom hook ergonomics and memo stability

- ☐ Hook name begins with `use` and describes what it gives, not how it works
- ☐ An exported result interface types the return value
- ☐ Object returns are wrapped in `useMemo`; callbacks in the return are wrapped in `useCallback`
- ☐ Every hook that owns a resource cleans it up internally, so consumers cannot leak it
- ☐ Generic hooks are genuinely generic; no `unknown` plus a cast at the call site
- ☐ Hooks in shared packages change additively; return-shape changes are treated as breaking
- ☐ Extraction justified by a second consumer or a separable concern, not by file length
- ☐ `useMemo` and `useCallback` each justified by a memoized consumer, a dependency array, or a measurement — no reflexive wrapping

Form performance and WCAG compliance

- ☐ Forms above roughly 15 fields use a headless form library rather than one controlled state object
- ☐ Typing latency measured on the target browser matrix; no perceptible lag at full field count
- ☐ Submission state modelled as a discriminated union; double submission structurally impossible
- ☐ Submit control disabled and relabelled from the same state value that guards the handler
- ☐ Every input has a programmatically associated `<label>` ; ids generated with `useId`
- ☐ Errors carry `role="alert"` , are linked with `aria-describedby` , and set `aria-invalid` from the same derived source as the message (SC 3.3.1)
- ☐ Validation messages name the problem and the remedy, never just "invalid"
- ☐ Related fields grouped in `<fieldset>` with a `<legend>`
- ☐ Step and progress position announced through a polite live region, not conveyed visually alone
- ☐ Focus moved to the new step heading on every transition, and not stolen on mount (SC 2.4.3)
- ☐ `0` and empty values render meaningful text, never a bare falsy leak
- ☐ Whole flow completed keyboard-only, verified by test

Route guards and navigation security

- ☐ Guards evaluate during render and return `<Navigate />` ; none redirect from an effect
- ☐ Auth state is a three-state union; `loading` never treated as `anonymous`
- ☐ Redirects use `replace` so the back button cannot re-enter a bounced route
- ☐ Return-to location preserved in navigation state for post-login restoration
- ☐ Protection applied structurally as a layout route, not repeated per page
- ☐ Client-side guards documented as UX only; server-side authorization verified independently
- ☐ Route params validated before use; `useParams` results are `string | undefined` and handled
- ☐ URL-derived state read from `useSearchParams` , never duplicated into `useState`
- ☐ Focus moved and route change announced on navigation
- ☐ Lazy route boundaries wrapped in `Suspense` with an accessible fallback, and in an error boundary for chunk-load failure (MOD-06.2, MOD-07.1)

RTL coverage thresholds

- ☐ Queries use `getByRole` / `getByLabelText`; no `container.querySelector`, no test IDs for user-facing elements
 - ☐ Every custom hook has `renderHook` tests including its cleanup path
 - ☐ Abort-on-unmount asserted by capturing the signal and checking `aborted`
 - ☐ Abort-on-dependency-change asserted, proving the race is structurally impossible
 - ☐ `expect(jest.getTimerCount()).toBe(0)` after unmount in every timer-owning hook's suite
 - ☐ Debounce asserted by call count under fake timers, not by waiting
 - ☐ Focus assertions cover every step and route transition
 - ☐ Guard tests assert both the redirect **and** that the protected request never fired
 - ☐ Error and empty paths tested alongside the happy path
 - ☐ Branch coverage at or above the project gate (90% for shared hooks), enforced in CI
 - ☐ No `act()` warnings in the suite output; warnings fail the build
-

Exit criteria for MOD-05. LAB-05 passes the full suite including both abort tests, the debounce call-count test, the focus-transition tests and the guard test asserting no privileged request; everything compiles under `strict` with zero `any`; all eight Tier A cards answer cxxold; and the learner can explain, without notes, why cleanup N-1 running before setup N is what prevents a tenant-switch race. Only then does MOD-06 begin.